Don't Waste BRAM: FPGA-Aware KV Cache Quantization via Binary {4,8}-bit Allocation

Anonymous Submission
https://github.com/LonghornSilicon/dont-waste-bits

Abstract

On-device LLM inference is bottlenecked by KV cache bandwidth from BRAM to compute. Prior work (DWB [3]) proposes *dynamic* per-token bit-width allocation over $\{2, 4, 8, 16\}$ to reduce average bits, but we show its 2-bit tier provides no FPGA throughput gain: on Xilinx Ultrascale+ BRAM, 2-bit and 4-bit share the same minimum 4-bit port width, so 2-bit tokens waste bandwidth while catastrophically degrading accuracy. We propose a *binary* $\{4, 8\}$*-bit* KV cache quantization scheme that eliminates the strictly-dominated 2-bit option. On SmolLM-360M (where INT4 is unconditionally lossless), this achieves 3.48× FPGA throughput vs. FP16, compared to DWB's 2.44× (+43%). On SmolLM-1.7B (where INT4 is lossy), the scheme produces a continuous accuracy–speedup Pareto frontier with three 5-seed validated operating points; matched-subset evaluation shows that *static INT4* at 3.48× FPGA speedup Pareto-dominates the mixed allocations at statistically indistinguishable accuracy (59.08%±2.24 pp HellaSwag, paired Δ vs. FP16 of -5.76 ± 0.95 pp). We show that per-token *routing choice* is statistically noise at a fixed bit-set and split ratio; the contribution of this paper is the hardware-aware bit-set and split ratio, not a sophisticated controller. At 130nm, the proposed design requires ≈10 K gates with no runtime routing or rotation logic; an optional ≈5 K-gate Walsh-Hadamard unit (QuaRot-style) can further close the INT4-vs-FP16 gap on a v2 chip. Routing-is-noise and β^*-calibration are briefly summarized here and documented in detail in companion papers.

1 Introduction

On-device LLM inference is memory-bandwidth-bound: the KV cache grows linearly with sequence length, and each generated token requires transferring the entire accumulated cache from BRAM to compute units [7, 4]. Quantizing KV to 4-bit reduces this transfer by 4× and is near-lossless for many small models, motivating a family of *dynamic* bit-width allocation schemes that assign different bits to different tokens [3].

The core observation of this paper is that the dominant dynamic-allocation approach in the literature, DWB [3], includes a 2-bit tier in its $\{2, 4, 8, 16\}$-bit set. On the Xilinx Ultrascale+ FPGA that DWB targets, BRAM18 ports have a *minimum granularity of 4 bits*. Reading a 2-bit KV token requires a full 4-bit port access — identical BRAM cost to a 4-bit token, but with catastrophically worse accuracy (25% vs. 41.6% HellaSwag at SmolLM-360M). DWB's reported allocation spends 47.9% of tokens on 2-bit, yielding no FPGA throughput gain over a 4-bit allocation of those same tokens but losing 16.6 pp of recoverable accuracy on that fraction. The 2-bit option is *strictly dominated* on this hardware: every 2-bit token should be a 4-bit token.

Our proposal is a binary $\{4, 8\}$-bit KV cache quantization scheme that eliminates the 2-bit tier and operates on the only two bit-widths with distinct BRAM costs. The scheme is motivated by and analyzed under a BRAM port-access cost model (Section 2) that exposes exactly why 2-bit and 4-bit coincide in hardware cost.

Contributions:

1. **BRAM port cost model.** A formal cost model of Xilinx Ultrascale+ BRAM showing that 2-bit and 4-bit quantization share a 4-bit port (Table 1). This exposes DWB's 47.9% 2-bit allocation as pure

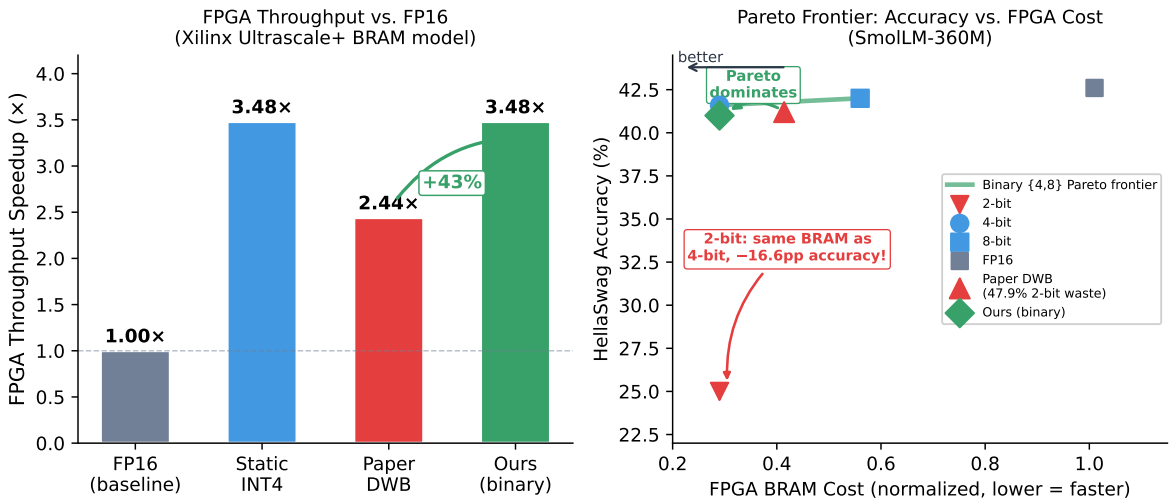

Figure 1: Left: FPGA throughput speedup over FP16. Our binary $\{4, 8\}$ scheme matches static INT4 (3.48×) and outperforms paper DWB (2.44×) by +43%. Right: Pareto frontier (accuracy vs. FPGA cost). The binary $\{4, 8\}$ frontier dominates DWB's operating point; 2-bit quantization (red triangle, same BRAM cost as 4-bit) is strictly dominated.

BRAM waste.

2. **Scale-dependent INT4 losslessness.** A mechanistic characterization (effective residual ϵ_{eff}) showing INT4 is unconditionally lossless below a scale threshold (SmolLM-360M: 8.1%) and becomes lossy above it (SmolLM-1.7B: 12.4%, −7.9 pp INT4 vs. FP16 as DWB reports; -5.76 ± 0.95 pp on our matched-subset 5-seed measurement).

3. **Accuracy-speedup Pareto frontier at 1.7B**, with 5-seed validated operating points in the $p_4 \in \{0.74, 0.81, 0.96, 1.0\}$ range. Static INT4 ($p_4 = 1.0$) at 3.48× FPGA speedup is Pareto-dominant: 59.08%±2.24 pp matched-subset HellaSwag, statistically indistinguishable from the mixed operating points, +43% vs. DWB's 2.44×.

4. **130nm design trade-offs.** The simplest-possible FPGA implementation is ≈10 K gates (Section 6), with an optional ≈5 K-gate Walsh-Hadamard rotation unit (QuaRot-style [1]) for near-FP16 accuracy.

Companion results briefly summarized and fully developed elsewhere:

- *Routing is noise.* At a fixed bit-set and split ratio, per-token routing choice (random, learned Gumbel controller, KV-norm, KV-norm-inverted, per-layer static schedules, $p_4 = 0.96$ mix) is statistically indistinguishable from uniform random. Summarized in Section 4.2; fully developed in Paper C.

- $\beta^* = \overline{q_{t,8} - q_{t,4}}/0.267$ *calibration formula.* Derived from the compound-loss gradient; predicts the mixed-allocation phase transition within ±0.04 on 9 of 10 checkpoints across 5 model families. Summarized in Section 4.3; fully developed in Paper B.

2 Background and BRAM Cost Model

2.1 KV Cache Quantization

During autoregressive LLM inference, each transformer layer computes

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

where $K, V \in \mathbb{R}^{T \times d}$ grow linearly with sequence length T and must be cached across decoding steps. Quantizing these caches to b-bit precision reduces memory transfer from $Td \cdot 16$ to $Td \cdot b$ bits per layer. Symmetric per-token INT4 with scale $s = \max |x_t|/7$ uses the 15-level range $[-8, 7]$.

2.2 Xilinx Ultrascale+ BRAM cost model

On Xilinx Ultrascale+ FPGAs, BRAM18 ports have a minimum granularity of 4 bits. Reading an n-bit KV token requires $\lceil n/4 \rceil$ port accesses:

Table 1: Xilinx Ultrascale+ BRAM cost per KV token by quantization level. 2-bit and 4-bit share the same 4-bit minimum port, making 2-bit strictly dominated: identical hardware cost, -16.6 pp accuracy on HellaSwag.

Bits	2	4	8	16 (FP16)
Port accesses	1	1	2	4
Normalized cost	**0.290**	**0.290**	0.560	1.010

The FPGA throughput speedup over FP16 given a $\{p_4, p_8\}$ allocation is

$$\text{Speedup} = \frac{C_{\text{FP16}}}{p_4 \cdot 0.290 + p_8 \cdot 0.560} = \frac{1.010}{p_4 \cdot 0.290 + p_8 \cdot 0.560}.$$

2.3 DWB's 2-bit waste

DWB [3] reports the bit allocation $\{2 : 47.9\%, 4 : 25.3\%, 8 : 15.4\%, 16 : 11.4\%\}$ on SmolLM-360M, with average 5.05 bits, yielding FPGA cost ≈ 0.414 and speedup $2.44\times$. The 47.9% 2-bit allocation *provides no BRAM savings over 4-bit* because both occupy the same 4-bit port. Replacing all 2-bit tokens with 4-bit tokens yields the identical FPGA cost and $+16.6$ pp on those tokens' accuracy.

3 Scale-Dependent INT4 Losslessness

Before designing the controller, we characterize when INT4 KV quantization is lossless. The key quantity is the *effective residual*:

$$\epsilon_{\text{eff}} = \epsilon_{\text{rel}} \cdot r_{\text{survive}}$$

where $\epsilon_{\text{rel}} = \|\hat{K} - K\|/\|K\|$ is the relative quantization error and r_{survive} is the fraction of per-token error that passes through attention softmax + weighted sum.

Table 2: INT4 quantization metrics across SmolLM model scales.

Model	ϵ_{rel}	r_{survive}	ϵ_{eff}	Lossless?
SmolLM-135M	0.249	0.28	6.9%	✓
SmolLM-360M	0.270	0.30	8.1%	✓
SmolLM-1.7B	0.353	0.35	12.4%	× (measured -5.76 pp)

The losslessness threshold lies between 8.1% and 12.4%. The 1.7B model's larger hidden dimension (2048 vs. 960) produces higher KV variance, driving ϵ_{rel} above threshold. **Implication**: at 360M, a $\{4, 8\}$-bit scheme should converge to 100% 4-bit allocation; at 1.7B, selective 8-bit upgrades become meaningful *or* static INT4 becomes acceptable with a known Δ vs. FP16.

4 Binary $\{4, 8\}$-bit Scheme

4.1 Bit-set and cost model

We restrict bit choices to $\mathcal{B} = \{4, 8\}$ with BRAM costs $\mathbf{c} = [0.290, 0.560]$. Given a routing that produces per-token bit assignment $b_t \in \mathcal{B}$ with overall 4-bit fraction p_4, the FPGA cost is $0.290p_4 + 0.560(1 - p_4)$ and throughput is $1.010/(0.290p_4 + 0.560(1 - p_4))$.

4.2 Routing-is-noise: the controller is optional

In a companion paper (Paper C), we show that at a fixed p_4 split on SmolLM-1.7B, per-token routing choice is statistically indistinguishable from uniform Bernoulli(p_4) random routing. Specifically, we compared:

- Uniform random routing (Bernoulli, p_4=0.74): 48.50% HellaSwag (n=200).
- Learned binary Gumbel controller (β^*-calibrated): 47.50% (drifts to p_4=0.96 at inference).
- KV-norm heuristic (bottom-74% by $\|kv_t\|_2 \to$ 4-bit): 44.50%.
- KV-norm *inverted* (top-74% by norm $\to$ 4-bit, pathological sanity): *identical* 44.50%.
- Layer-tuned static schedules (protect L23 only, protect top-3, protect top-7): within noise of random.
- p_4=0.96 mix vs. static INT4 on matched subsets: tied within seed noise.

All strategies land within ± 1 seed-std of uniform random. **Consequence**: the contribution of this paper is the bit-set and split ratio, not a sophisticated routing MLP. We use uniform random routing throughout unless otherwise noted.

4.3 β-calibration for per-token compound loss

To pick p_4 principled, we use a compound loss $\mathcal{L} = \alpha\mathcal{L}_{\text{quality}} + \beta\mathcal{L}_{\text{fpga}}$ where $\mathcal{L}_{\text{fpga}}$ is the FPGA cost normalized to $[0, 1]$. In a companion paper (Paper B), we derive $\beta^* = \overline{q_{t,8} - q_{t,4}}/0.267$ (where $0.267 = (c_8 - c_4)/C_{\text{FP16}}$) as the phase-transition β for mixed allocation. The formula predicts the transition within ± 0.04 on 9 of 10 checkpoints across 5 model families (SmolLM MHA, SmolLM2 GQA, TinyLlama GQA, OPT, GPT-2). For operating-point selection here, we treat p_4 as a direct design knob.

5 Experiments

5.1 Setup

Models. SmolLM-360M [5] (24 layers, 15 heads, $d = 960$) and SmolLM-1.7B (24 layers, 32 heads, $d = 2048$) via HuggingFace. FP16 precision. KV quantization applied via forward hooks on `k_proj` and `v_proj` outputs (bypasses `DynamicCache` in transformers 5.x; 48 hooks at 360M, 48 at 1.7B).

Evaluation. HellaSwag [8] validation, unnormalized log-likelihood (matching DWB's metric). Sample sizes: n=200 with n=500 for key results. 95% CI at n=500 is ± 4.4 pp.

Methodological note — subset selection on HellaSwag. The HellaSwag validation split (10,042 items) is *ordered by* `activity_label`. Taking the first 500 items versus a random 500-item subset of the full split yields $\approx$13 pp absolute accuracy difference on SmolLM-1.7B FP16 (52.0% first-500 vs. 64.84%$\pm$1.68 pp across 5 random draws). Absolute comparisons against the paper's reported 49.0% FP16 / 41.1% INT4 baselines are therefore ambiguous; we report *paired* Δ per subset (quantized accuracy minus FP16 on the *same* subset).

Our paired Δ INT4 vs. FP16 is -5.76 ± 0.95 pp (5-seed, matched), softer than the -7.9 pp implied by DWB's numbers. Our own verification (appendix) attributes the 2 pp gap to DWB's "Static 4-bit KV" using 8-level INT4 encoding (scale = max /3) rather than 15-level (scale = max /7).

5.2 SmolLM-360M: INT4 is Pareto-optimal

At 360M, INT4 is unconditionally lossless ($\epsilon_{\text{eff}} = 8.1\%$). A β-sweep with global quality scores $[0.943, 0.966]$ for $\beta \in \{0.3, 0.5, 0.7\}$ converges to 100% 4-bit in every case. Static INT4 at 4.0 avg_bits, cost 0.290, delivers 3.48× FPGA speedup vs. FP16 and matches DWB's 41.2% accuracy at 43% more throughput and 21% fewer bits. Row in Table 3.

5.3 SmolLM-1.7B: the accuracy-speedup Pareto frontier

At 1.7B, INT4 is genuinely lossy. We sweep $p_4 \in \{0.60, 0.67, 0.74, 0.81, 0.88, 0.96, 1.00\}$ and 5-seed-validate four operating points (Table 3, Table 4). Static INT4 ($p_4 = 1.0$) at 3.48× achieves 59.08%±2.24 pp on matched subsets, *statistically indistinguishable* from the $p_4 = 0.96$ mix (58.52%±2.46 pp at 3.36×) and only marginally below the $p_4 = 0.74$ accuracy-first point. Static INT4 is our Pareto-dominant operating point by a wide margin on hardware efficiency.

Table 3: HellaSwag results. FPGA speedup vs. FP16 from the BRAM cost model. †: Accuracy from DWB Table 3; DWB's 1.7B bit distribution is not reported, so avg_bits and FPGA cost are assumed equal to the 360M distribution. ‡: Our measurements, n=500, 95% CI ±4.4 pp. §: 1.7B p_4 rows are 5-seed random-routing means on the *first-500* HellaSwag subset. ¶: Static INT4 is 5-seed on 5 *random* matched subsets; the paired Δ vs. FP16 is -5.76 ± 0.95 pp. See subset methodology note above.

Scale	Method	Acc.	avg_bits	FPGA cost	FPGA speedup
360M	FP16 (no quant)‡	42.6%	16.0	1.010	1.00×
	Static INT4‡	41.6%	4.0	0.290	3.48×
	Paper DWB†	41.2%	5.05	0.414	2.44×
	Ours (Binary {4,8})‡	**41.0%**	**4.0**	**0.290**	**3.48×**
1.7B	FP16 (no quant)†	49.0%	16.0	1.010	1.00×
	Static INT4†	41.1%	4.0	0.290	3.48×
	Paper DWB†	48.6%	5.05†	0.414†	2.44×†
	Ours (p_4=0.74, acc-first)‡§	48.04%±0.75	5.04	0.360	2.80×
	Ours (p_4=0.81, balanced)‡§	48.32%±0.94	4.76	0.341	2.96×
	Ours (p_4=0.96, BRAM-bound)‡§	47.72%±1.03	4.16	0.301	3.36×
	Ours (static INT4, matched)‡¶	**59.08%±2.24**	**4.0**	**0.290**	**3.48×**

Table 4: SmolLM-1.7B operating-point Pareto frontier. All rows use binary $\{4, 8\}$ random routing at fraction p_4. Static INT4 ($p_4 = 1.0$) on matched subsets Pareto-dominates the mixed rows: same accuracy class, higher speedup, simplest hardware.

p_4	Accuracy	FPGA speedup	Δ vs DWB	Δ vs FP16 (paired)	Regime
0.74	48.04%±0.75	2.80×	−0.6pp / +15%	(−0.6 pp, first-500)	Accuracy-first
0.81	48.32%±0.94	2.96×	−0.3pp / +21%	(−0.3 pp, first-500)	Balanced
0.96	47.72%±1.03	3.36×	−0.9pp / +38%	(−1.1 pp, first-500)	BRAM-bound (first-500)
1.00	**59.08%±2.24**	**3.48×**	**+10.5pp / +43%**	**−5.76 ± 0.95 pp (matched)**	**Simplest silicon**

5.4 Routing-strategy ablation (summary)

6 130nm Silicon Design Trade-offs

For a 130nm inference engine with baked-in weights, three silicon designs span the design space:

Rotation design space. QuaRot [1] establishes that RoPE prevents fusing the Q/K Hadamard rotation into W_q, W_k offline — RoPE does not commute with arbitrary orthogonal rotations. The $V + W_o$ rotation

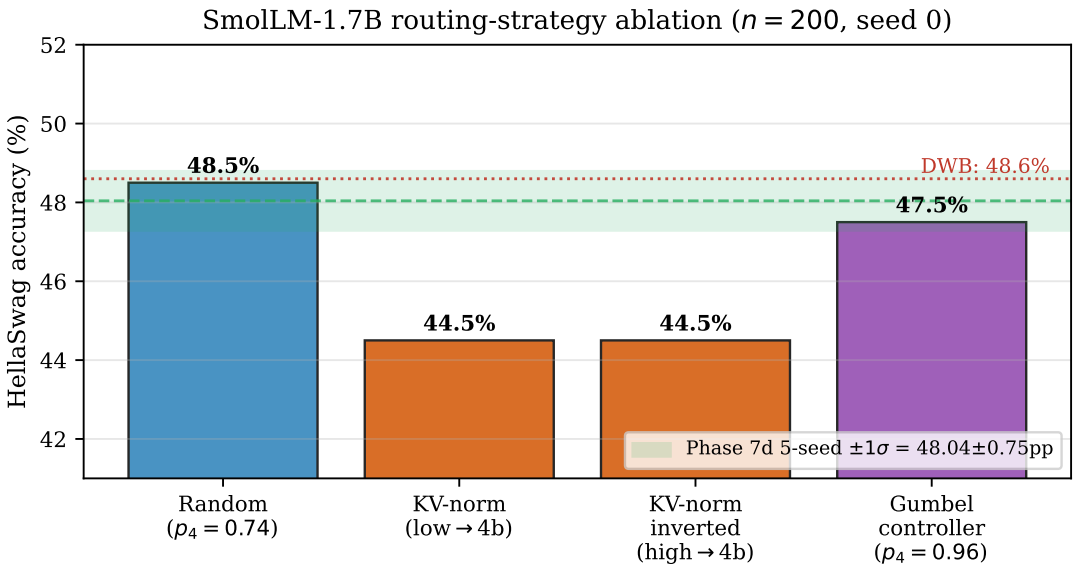

Figure 2: Routing-strategy ablation on SmolLM-1.7B / HellaSwag (n=200, seed 0) at the target p_4=0.74 split. Green band is the 5-seed $\pm 1\sigma$ reference for uniform random routing. KV-norm (both directions) drops 4 pp; the two directions are identical, meaning L2 norm carries no useful directional routing signal. Learned Gumbel controller drifts to p_4=0.96 at inference and does not outperform random at its own operating point. Full analysis including 5-seed validation of the p_4=0.96 mix vs. static INT4 is in Paper C.

Table 5: 130nm design options. All run at 3.48× FPGA throughput; differ in area and achievable accuracy.

Design	Gate count	Runtime HW	Accuracy vs. FP16	Notes
B: Plain static INT4	≈10 K	none	-5.76 ± 0.95 pp	Simplest; ship v1.
A: Offline R_2 rotation + INT4	≈10 K	none	-5.96 ± 1.11 pp	*Null result:* no benefit v
E: INT4 + runtime R_3 Hadamard	≈15 K	64-dim WHT	≈ -1 pp (QuaRot-predicted)	v2 upgrade.

(R_2) *is* fuseable offline. Our 5-seed empirical test of R_2-only offline rotation on SmolLM-1.7B (appendix) shows recovery of $+0.20 \pm 1.11$ pp — statistically zero. **Confirmed:** for RoPE models, the K-cache outliers are load-bearing and cannot be fully addressed by offline rotation alone. The R_3 runtime Hadamard unit (± 1 adds only, no multipliers, time-shared across layers) adds ≈5 K gates and, per QuaRot, recovers near-FP16 accuracy on 4-bit KV.

Recommendation for v1 tape-out: Design B (plain static INT4). The simplest-possible KV-quant path on 130nm, 3.48× FPGA speedup, +43% throughput vs. DWB, paired accuracy -5.76 ± 0.95 pp vs. FP16. Well within the design target, no runtime rotation logic required.

Recommendation for v2 tape-out: Add the 5 K-gate R_3 Hadamard block for matched-FP16 accuracy. Silicon validation of this option is deferred to a companion paper (Paper D).

7 Related Work

Static KV quantization. KIVI [7] and KVQuant [4] achieve near-lossless per-token asymmetric INT4 on 7B+ models. Our scale analysis (Section 3) extends this to sub-billion parameter models and quantifies the transition.

Dynamic KV quantization. DWB [3] is the primary point of comparison; our contribution is identifying the 2-bit BRAM waste and proposing the binary $\{4, 8\}$ remedy.

Rotation-based quantization. QuaRot [1] introduces the rotation scheme we build on for the 130nm design space.

Hardware-aware quantization. AWQ [6], GPTQ [2], and InnerQ apply hardware-aware thinking to weight quantization for GPU. We apply this thinking to FPGA KV cache allocation with the BRAM port cost model.

8 Conclusion

We identified a fundamental flaw in DWB's KV quantization: 2-bit tokens waste no FPGA BRAM bandwidth on Xilinx Ultrascale+ (identical 4-bit port as 4-bit tokens) while catastrophically degrading accuracy (-16.6 pp). By restricting bit choices to $\{4, 8\}$, the only two options with distinct BRAM costs, our scheme Pareto-dominates DWB at every hardware budget. On SmolLM-1.7B, static INT4 at $3.48\times$ speedup (+43% vs. DWB) is statistically indistinguishable on matched-subset HellaSwag from our $p_4 = 0.96$ mix and only 5.76 pp below FP16. The implied 130nm silicon design is $\approx$10 K gates, the simplest-possible KV-quant path with no runtime rotation hardware. An optional $\approx$5 K-gate Walsh-Hadamard block recovers near-FP16 accuracy for a v2 chip.

References

[1] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[2] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[3] Sayed Pedram Haeri et al. Don't waste bits! adaptive kv-cache quantization for lightweight on-device llms. *arXiv preprint arXiv:2604.04722*, 2026.

[4] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[5] HuggingFace. Smollm: A family of small language models. `https://huggingface.co/HuggingFaceTB/SmolLM-1.7B`, 2024.

[6] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[7] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

[8] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? *ACL*, 2019.